

Documentación Técnica: App de Control de Territorios

1. Visión General

La aplicación de Control de Territorios es una herramienta en tiempo real diseñada para coordinar equipos en campo. Permite la visualización de mapas, el marcado de manzanas, y el dibujo de áreas de interés, sincronizando el estado inmediatamente en los dispositivos de todos los usuarios conectados.

2. Arquitectura del Sistema

El sistema sigue una arquitectura Cliente-Servidor (Full-Stack) empaquetada en un único entorno para facilitar su despliegue y portabilidad.

2.1 Backend (Servidor)

El backend es el "cerebro" del sistema. Su función es despachar la página web a los usuarios y mantener el control del estado en tiempo real.

- **Tecnología Base:** Node.js
- **Razón de uso:** Node.js permite usar JavaScript en el servidor, lo que unifica el lenguaje de programación en todo el proyecto. Es extremadamente rápido para manejar múltiples conexiones simultáneas gracias a su arquitectura asíncrona (ideal para aplicaciones en tiempo real).

Frameworks y Librerías del Servidor:

- **Express.js:**
 - *¿Qué hace?* Es un framework web para Node.js.
 - *¿Por qué se usó?* Simplifica enormemente la creación del servidor HTTP, el manejo de rutas y la entrega de los archivos estáticos (el código del Frontend ya compilado).
 - **Socket.IO:**
 - *¿Qué hace?* Crea una conexión persistente (WebSockets) entre el navegador del usuario y el servidor.
 - *¿Por qué se usó?* Es la clave de la aplicación. En lugar de que el celular tenga que "preguntar" cada 5 segundos si alguien modificó el mapa, Socket.IO mantiene un túnel abierto. Cuando un compañero marca una manzana, el servidor le "avisa" instantáneamente a todos los demás.
 - **fs (File System) y path:**
 - *¿Qué hacen?* Son módulos nativos de Node.js.
 - *¿Por qué se usaron?* Se encargan de leer y escribir de forma eficiente los archivos `.json` (como el progreso de los mapas en `data.json` y las coordenadas de los territorios en la carpeta `mapas/`).
-

3. Frontend (Interfaz de Usuario)

Es lo que ven los usuarios (la web en el navegador/celular). Está diseñada para ser rápida, interactiva y reactiva.

3.1 Tecnologías Base

- **React.js:**
 - *¿Qué hace?* Es una librería para construir interfaces de usuario basada en componentes.
 - *¿Por qué se usó?* A diferencia de una página web tradicional, React permite que la pantalla se actualice solo en las partes que cambian (por ejemplo, pintar un lado de la manzana de verde sin recargar toda la página). Esto hace que la app se sienta rápida como una aplicación nativa instalada en el celular.
- **Vite:**
 - *¿Qué hace?* Es el empaquetador y servidor de desarrollo.
 - *¿Por qué se usó?* Es mucho más rápido y moderno que alternativas antiguas como *Create React App*. Toma todo nuestro código React y lo comprime en archivos súper ligeros que Express puede enviar a los usuarios rápidamente.

3.2 Librerías de Mapas

- **Leaflet.js:**

- *¿Qué hace?* Es la principal librería de código abierto para mapas interactivos.
 - *¿Por qué se usó?* Es completamente gratuita y no requiere tarjetas de crédito ni llaves API como Google Maps. Funciona perfectamente con OpenStreetMap, que proporciona fotografías satelitales globales libres de costo.
- **React-Leaflet:**
 - *¿Qué hace?* Traduce las funciones crudas de Leaflet al ecosistema de componentes de React.
 - *¿Por qué se usó?* Permite escribir código mucho más limpio. En lugar de usar funciones matemáticas complejas para dibujar polígonos, nos permite declarar componentes como `<Polygon>` o `<Marker>` directamente en el código de la pantalla.
 - **Leaflet.draw (Usado en el Editor):**
 - *¿Qué hace?* Añade barras de herramientas para que el usuario pueda dibujar polígonos y líneas libremente sobre el mapa.
 - *¿Por qué se usó?* Proporciona la interfaz gráfica exacta (lápices, polígonos, borradores) necesaria para que cualquiera pueda trazar los límites del territorio sin ser programador.
-

4. Despliegue y Portabilidad

La aplicación fue diseñada para poder ejecutarse tanto en una computadora vieja con Windows (vía scripts de PowerShell portables) como en la nube.

- **Docker y Docker Compose:**
 - *¿Qué hacen?* Meten toda la aplicación (servidor, base de datos de archivos y frontend) dentro de un "contenedor" hermético.
 - *¿Por qué se usó?* Garantiza que si la aplicación funciona en tu PC, funcionará exactamente igual en AWS, sin importar qué versión de Linux o qué librerías tenga instaladas la máquina. Esto elimina el clásico problema de "en mi computadora sí funcionaba".
-

Generado automáticamente para el equipo de Control de Territorios.